# LLD Phase 5 — PDF 5

Logging Framework · Hotel Booking · Task Scheduler · Food Delivery | Requirements first, hints in the middle, full solutions at the back

> **Colour key. Purple = the interviewer's words · Blue = a note or hint · Red = a common mistake · Green = a rule to remember · Amber = how to say it in an interview.**

### Contents


**Before you start** — the loop · the rush helper · time boxes · warm-up: two PDF 4 designs from memory, then 12 quick ones

**Problem 1** — Logging Framework (small)

**Problem 2** — Hotel Booking (core)

**Problem 3** — Task Scheduler (core)

**Problem 4** — Food Delivery (heavy)

**Hints** — three levels per problem: a nudge, the shape, the key decision

**Solutions 1–4** — clarifying questions · Part A model answer · what to write first · full code · Part C answers · common mistakes · 10-point checklist

**Back pages** — warm-up answers · mistake log · re-do schedule


## Before you start

Same format as PDFs 3 and 4: the interviewer's words in purple first (write your own questions before reading on), interview-sized solutions, and a "write this first" order in each. The loop stays the same: Part A on paper, one hint level only after 15 minutes stuck, Part B until the output matches exactly, Part C out loud, then the solution and its checklist.

| New in PDF 5 | What it means for you |
|---|---|
| **Hierarchies of settings** | Loggers inherit their level from their parent ("com.app.pay" from "com.app"). The same "walk up until you find a value" idea appears in config systems and permissions. |
| **Two queues, one clock** | The task scheduler keeps "not due yet" and "due, waiting for a worker" in two different orders. That's how `ScheduledThreadPoolExecutor` and cron-like systems work underneath. |
| **A whole product** | Food Delivery is the first problem with several actors (customer, restaurant, delivery partner), an order life cycle, assignment, a waiting queue and notifications all at once. Draw before you code. |

### The rush helper (given, same as before)

```
// Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
static int rush(int n, IntPredicate task) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(n);
    CountDownLatch go = new CountDownLatch(1);
    List<Future<Boolean>> results = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        int id = i;
        results.add(pool.submit(() -> { go.await(); return task.test(id); }));
    }
    go.countDown();
    int trues = 0;
    for (Future<Boolean> f : results) if (f.get()) trues++;
    pool.shutdown();
    return trues;
}
```

Used by Hotel Booking and Food Delivery. It needs `import java.util.concurrent.*;` and `import java.util.function.IntPredicate;`, and `main` must declare `throws Exception`.

| Problem | Size | Time box | Format |
|---|---|---|---|
| 1. Logging Framework | small | 45 min | 10 design + 35 code |
| 2. Hotel Booking | core | 75 min | 20 design + 55 code |
| 3. Task Scheduler | core | 75 min | 20 design + 55 code |
| 4. Food Delivery | heavy | 100 min | 25 design + 75 code |

### Warm-up (15 minutes)

**Part 1 — from memory.** Don't open PDF 4. Write on paper, then check against the back pages.

| | |
|---|---|
| **W1** | **Cache with TTL** (5 min). Lazy and active expiry: what does each do, and why do real caches use both? Which two structures give "least recently used" and "soonest to expire"? How does a test control time? When the cache is full, what goes first? |
| **W2** | **Meeting Room Scheduler** (5 min). Write the clash condition for [s1, e1) and [s2, e2). Which single TreeMap lookup finds a clash, and why is one enough? What is locked during a booking? How does "any room" stay safe without locking every room? |

**Part 2 — name the pattern** (5 min). All from one app: an online mock-test platform for JEE and NEET. Name the pattern, or say "nothing" and what you'd use instead.

1. Scoring differs by exam: +4/−1 for JEE, no negative marks for school tests, partial marks for some papers. New exams keep arriving.
2. A test attempt goes NOT_STARTED → IN_PROGRESS → SUBMITTED → EVALUATED. What the student, the proctor and the evaluator can each do depends on the status.
3. The proctoring vendor's SDK offers `startSession(String token, int camId)`. Our code calls `Proctor.watch(Attempt)` everywhere.
4. On submission: email a receipt, start evaluation, update the leaderboard. More reactions will come.
5. Re-evaluation requests: a teaching assistant handles up to 2 marks, the subject head up to 10, above that the exam controller.
6. Registering for an exam: verify ID, take payment, allot a centre, send the admit card. The website, the app and partner coaching centres all do this.
7. A paper has sections, which have questions, some of which have sub-parts. The total marks are needed at every level.
8. Students who registered but haven't paid within 3 days get a reminder.
9. The same diagram image appears in 50,000 students' papers; it should be stored once.
10. Past results come from a slow old system. Keep answers for an hour, and only the student or their teacher may see them.
11. An answer option has a label (A to D) and a text.
12. A mock test has a name and a duration (required) and many optional settings: shuffle, section timers, negative marking, calculator. It never changes once published.

# Problem 1 — Logging Framework

**Small** · 45 minutes (10 design + 35 code) · machine-coding format

> **The interviewer says:** "Design a logging library like Log4j or SLF4J. Messages have levels, and each part of the code can be set to a different level. Logs go to several destinations, such as the console and a file, each with its own format. Adding a new destination shouldn't touch the logging code."

### After your questions, the requirements are

| | |
|---|---|
| **R1** | Levels, from least to most severe: DEBUG, INFO, WARN, ERROR. |
| **R2** | Loggers have dotted names ( `com.app.payment` ). Asking twice for the same name gives the same logger. |
| **R3** | A logger can have its own level. If it doesn't, it uses its nearest ancestor's: `com.app.payment` → `com.app` → `com` → `root` . The root's level starts as INFO. |
| **R4** | A message below its logger's level is dropped (counted as filtered). Otherwise it goes to every appender whose own minimum level it meets. |
| **R5** | Appenders: a **console** appender writes each line at once; a **buffer** appender (standing in for a file) collects lines and writes them all when it has N, or on FLUSH. Every line an appender writes starts with its name and `\|` . |
| **R6** | Each appender has a pattern: `%t` time, `%l` level, `%c` logger name, `%m` message; other text stays as written. |
| **NF1** | New destinations (a real file, Kafka) and new formats (JSON) must not change the logger code. |
| **NF2** | Many threads log at once, and logging must not slow the application down (Part C asks). |

**Out of scope:** config files, appenders per logger, log rotation, structured fields.

### Commands and exact output

| Command | Output |
|---|---|
| `TIME hh:mm:ss` | `Time 09:00:00` (the time stamped on later messages) |
| `LEVEL logger level` | `Level of com.app: DEBUG` ( `root` is the root logger) |
| `CONSOLE name level pattern…` | `Appender out: console, INFO and above` |
| `BUFFER name level size pattern…` | `Appender file: buffer of 3, DEBUG and above` |
| `LOG logger level message…` | whatever the appenders write, e.g. `out\| [INFO] com.app.payment - Payment started` ; nothing if filtered |
| `FLUSH` | the buffered lines; `Nothing to flush` if there are none |
| `STATS` | `Logged 7, filtered 2` |
| `(error)` | `Unknown level TRACE` |

### Part A — design

| | |
|---|---|
| 1 | Classes and who owns what: level, logger, the level settings, appender, format, a log event. |
| 2 | R3: how do you find a logger's effective level? Where does that logic live? |
| 3 | NF1: what varies, and which patterns does that suggest for destinations and for formats? |
| 4 | Many tutorials use Chain of Responsibility for levels (a DEBUG handler, then INFO…). Would you? Why or why not? |

## Part B — code

**input.txt**

```
TIME 09:00:00
CONSOLE out INFO [%l] %c - %m
BUFFER file DEBUG 3 %t %l %c: %m
LOG com.app.payment INFO Payment started
LOG com.app.payment DEBUG Card checked
LEVEL com.app DEBUG
LOG com.app.payment DEBUG Card checked
LOG com.app.payment WARN Retry 1 of 3
TIME 09:00:05
LEVEL com.app.payment ERROR
LOG com.app.payment WARN Retry 2 of 3
LOG com.app.orders DEBUG New order 42
LOG com.other INFO Hello from other
LOG com.app.payment.upi ERROR UPI is down
LEVEL com.app TRACE
LOG com.app.orders INFO Order 42 packed
FLUSH
FLUSH
STATS
```

**expected output**

```
Time 09:00:00
Appender out: console, INFO and above
Appender file: buffer of 3, DEBUG and above
out| [INFO] com.app.payment - Payment started
Level of com.app: DEBUG
out| [WARN] com.app.payment - Retry 1 of 3
file| 09:00:00 INFO com.app.payment: Payment started
file| 09:00:00 DEBUG com.app.payment: Card checked
file| 09:00:00 WARN com.app.payment: Retry 1 of 3
Time 09:00:05
Level of com.app.payment: ERROR
out| [INFO] com.other - Hello from other
out| [ERROR] com.app.payment.upi - UPI is down
file| 09:00:05 DEBUG com.app.orders: New order 42
file| 09:00:05 INFO com.other: Hello from other
file| 09:00:05 ERROR com.app.payment.upi: UPI is down
Unknown level TRACE
out| [INFO] com.app.orders - Order 42 packed
file| 09:00:05 INFO com.app.orders: Order 42 packed
Nothing to flush
Logged 7, filtered 2
```

## Part C — pressure questions

| | |
|---|---|
| 1 | **Performance.** "Writing to a file on every log call is slowing our API down." What do you change? |
| 2 | **Change.** "Each logger should be able to have its own appenders, and still send to its parents' appenders." |
| 3 | **Failure.** "The disk is full and the file appender throws." What must happen to the application? |
| 4 | **Performance.** "`log.debug("user " + user)` builds the string even when DEBUG is off." |
| 5 | **Defend it.** "Make LogManager a Singleton, like every logging library." |

# Problem 2 — Hotel Booking

**Core** · 75 minutes (20 design + 55 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Design a hotel booking system, like a small OYO or MakeMyTrip for one hotel. Rooms are of different types. Guests search for free rooms for their dates, book, and cancel. A room must never be booked twice for the same night. Prices go up at weekends, and the refund depends on how early you cancel."

### After your questions, the requirements are

| | |
|---|---|
| **R1** | Room types and base prices per night: STANDARD Rs 2000, DELUXE Rs 3500, SUITE Rs 6000. Rooms have unique numbers. |
| **R2** | Dates are `yyyy-mm-dd`. A stay from check-in to check-out covers the nights check-in … check-out − 1, so one guest can leave on the day the next arrives. Check-out must be after check-in. |
| **R3** | Friday and Saturday nights cost 20% more (`price * 120 / 100`). A stay's price is the sum of its nights. |
| **R4** | Search shows, for each type, how many rooms are free for every night of the stay and what the stay would cost. |
| **R5** | Booking takes a type, not a room: the guest gets the lowest-numbered room of that type that is free for every night. Bookings get IDs B1, B2, … (failed attempts don't take a number). Refused, in this order, if: the type is unknown, a date is invalid, check-out isn't after check-in, or no room of that type is free. |
| **R6** | Cancelling gives today's date. Refund: 100% at least 7 days before check-in, 50% at 2–6 days, 0% the day before. On or after check-in it's refused. A booking can be cancelled once. |
| **NF1** | Pricing rules (festival season, long-stay discounts) and refund rules (non-refundable rates) will change; neither should touch the booking code. |
| **NF2** | Many guests book at the same moment; one room-night is never sold twice (RUSH). Different rooms must not wait for each other. |

**Out of scope:** several hotels, payment, check-in desk operations, changing a booking's dates.

### Commands and exact output

| Command | Output |
|---|---|
| `ROOM number type` | `Room 101 (STANDARD) added` |
| `SEARCH check-in check-out` | one line per type, in the order above: `DELUXE: 2 free, Rs 11900` or `DELUXE: none free` |
| `BOOK guest type check-in check-out` | `B1: Asha, room 201 (DELUXE), 2026-12-24 to 2026-12-27, 3 nights, Rs 11900` (`1 night` for one) |
| `CANCEL booking today` | `B2 cancelled 3 days before check-in, refund Rs 3850 (50%)` (`1 day` for one) |
| `RUSH type check-in check-out n` | n guests book that type and those dates at the same moment: `Rush: 2 booked, 48 refused` |

| Error | Output |
|---|---|
| `number taken / unknown type / bad date` | `Room 101 already exists` · `Unknown room type PENTHOUSE` · `Invalid date 2026-13-01` |
| `dates in the wrong order / nothing free` | `Check-out must be after check-in` · `No DELUXE room free for 2026-12-26 to 2026-12-27` |
| `unknown / already cancelled / too late` | `Unknown booking B9` · `B1 is already cancelled` · `Cannot cancel on or after check-in` |

### Part A — design

| | |
|---|---|
| 1 | Entities and who owns what: hotel, room, room type, booking, the price of a night, the refund rule. |
| 2 | How do you check that a room is free for a whole date range quickly? (You've seen this before.) |
| 3 | NF1: where do pricing and refunds live? |
| 4 | NF2: the guest asks for a *type*. What is the one indivisible step, and what do you lock? |

### Part B — code

**input.txt**

```
ROOM 101 STANDARD
ROOM 102 STANDARD
ROOM 201 DELUXE
ROOM 202 DELUXE
ROOM 301 SUITE
ROOM 101 SUITE
ROOM 401 PENTHOUSE
SEARCH 2026-12-24 2026-12-27
BOOK Asha DELUXE 2026-12-24 2026-12-27
BOOK Ravi DELUXE 2026-12-26 2026-12-28
BOOK Meera DELUXE 2026-12-26 2026-12-27
BOOK Kiran DELUXE 2026-12-27 2026-12-29
BOOK Kiran SUITE 2026-12-27 2026-12-27
BOOK Kiran SUITE 2026-13-01 2026-13-02
SEARCH 2026-12-26 2026-12-27
BOOK Neha STANDARD 2026-12-24 2026-12-25
CANCEL B1 2026-12-10
CANCEL B1 2026-12-11
CANCEL B2 2026-12-23
CANCEL B4 2026-12-23
CANCEL B3 2026-12-28
CANCEL B9 2026-12-01
SEARCH 2026-12-26 2026-12-27
RUSH DELUXE 2027-01-08 2027-01-10 50
SEARCH 2027-01-08 2027-01-10
```

**expected output**

```
Room 101 (STANDARD) added
Room 102 (STANDARD) added
Room 201 (DELUXE) added
Room 202 (DELUXE) added
Room 301 (SUITE) added
Room 101 already exists
Unknown room type PENTHOUSE
STANDARD: 2 free, Rs 6800
DELUXE: 2 free, Rs 11900
SUITE: 1 free, Rs 20400
B1: Asha, room 201 (DELUXE), 2026-12-24 to 2026-12-27, 3 nights, Rs 11900
B2: Ravi, room 202 (DELUXE), 2026-12-26 to 2026-12-28, 2 nights, Rs 7700
No DELUXE room free for 2026-12-26 to 2026-12-27
B3: Kiran, room 201 (DELUXE), 2026-12-27 to 2026-12-29, 2 nights, Rs 7000
Check-out must be after check-in
Invalid date 2026-13-01
STANDARD: 2 free, Rs 2400
DELUXE: none free
SUITE: 1 free, Rs 7200
B4: Neha, room 101 (STANDARD), 2026-12-24 to 2026-12-25, 1 night, Rs 2000
B1 cancelled 14 days before check-in, refund Rs 11900 (100%)
B1 is already cancelled
B2 cancelled 3 days before check-in, refund Rs 3850 (50%)
B4 cancelled 1 day before check-in, refund Rs 0 (0%)
Cannot cancel on or after check-in
Unknown booking B9
STANDARD: 2 free, Rs 2400
DELUXE: 2 free, Rs 4200
SUITE: 1 free, Rs 7200
Rush: 2 booked, 48 refused
STANDARD: 2 free, Rs 4800
DELUXE: none free
SUITE: 1 free, Rs 14400
```

**Check the prices by hand.** 2026-12-24 is a Thursday, so a DELUXE stay from the 24th to the 27th is Thursday 3500 + Friday 4200 + Saturday 4200 = Rs 11900. Kiran gets room 201 from the 27th because Asha checks out that morning.

### Part C — pressure questions

| | |
|---|---|
| 1 | **Change.** "Diwali week costs 50% more, and stays of 7+ nights get 10% off." Where does it go? |
| 2 | **Change.** "Hold the room for 10 minutes while the guest pays." You've done this before: what's the same, what's different? |
| 3 | **Scale.** "Many hotels, bookings in a database, several servers." |
| 4 | **Change.** "Overbooking: sell up to 105% of rooms, because some guests never show up." How would your design allow it? |
| 5 | **Defend it.** "Let the guest pick the room number. Assigning automatically is extra work." |

# Problem 3 — Task Scheduler

**Core** · 75 minutes (20 design + 55 code) · machine-coding format

> **The interviewer says:** "Design a task scheduler, something like cron or Java's ScheduledExecutorService. Tasks can run once at a given time, or repeat. There's a fixed pool of workers. When more tasks are due than there are workers, the more important ones go first. Tasks can be cancelled."

### After your questions, the requirements are

| | |
|---|---|
| **R1** | There are W workers, W1, W2, .... Time is in whole seconds from 0, and RUN_UNTIL moves it forward, second by second. |
| **R2** | A task has a unique name, a first due time, a duration (at least 1 second) and a priority, where 1 is the most urgent. A repeating task also has an interval (at least 1 second). |
| **R3** | Each second t, in this order: (1) runs that end at t finish (workers in order); a repeating task is next due at t + interval (a fixed delay after it finishes); (2) every task due at or before t starts waiting; (3) each idle worker, in order, takes the waiting task with the most urgent priority, then the earliest due time, then the one created first. |
| **R4** | A task scheduled with a due time already in the past waits from the next second processed. |
| **R5** | Cancelling removes a task from the waiting and future lists. If it's running, that run finishes, but nothing more is scheduled. |
| **R6** | Status shows the time, each worker (idle, or the task and when it ends), the waiting tasks in the order they'll be taken, and the next task due. |
| **NF1** | The rule for "which waiting task goes first" will change (for example, ageing: a task that has waited long gets more urgent). |
| **NF2** | In production the workers are real threads (Part C asks); the test uses this simulated clock so the output never changes. |

**Out of scope:** cron expressions, retries, dependencies between tasks, tasks that run longer than planned.

### Commands and exact output

| Command | Output |
|---|---|
| `WORKERS n` | `Scheduler ready: 2 workers` |
| `ONCE name at duration priority` | `backup: once at t=0, 5s, priority 2` |
| `EVERY name first interval duration priority` | `report: every 4s from t=1, 2s, priority 1` |
| `RUN_UNTIL t` | one line per event: `t=2: email finished on W1` · `t=2: report started on W1` |
| `CANCEL name` | `Cancelled report`, plus `(the current run will finish)` if it's running |
| `STATUS` | `t=1`, then `W1: email until t=2` or `W2: idle`, then `Waiting: report, thumbnails` and `Next: ping at t=16` when there are any |
| `(errors)` | `Task email already exists` · `Duration must be at least 1` · `Interval must be at least 1` · `Unknown task report` · `Time cannot go back` |

### Part A — design

| | |
|---|---|
| 1 | Tasks live in two groups: not yet due, and due but waiting for a worker. Which order does each group need, and which structure gives it? |
| 2 | Write the three steps of one second. Why does the order of the steps matter? |
| 3 | Fixed rate (every 4 s by the clock) or fixed delay (4 s after each run finishes)? What goes wrong with fixed rate when a run is slow? |
| 4 | NF1: where does the "which goes first" rule live? |

## Part B — code

**input.txt**

```
WORKERS 2
ONCE backup 0 5 2
ONCE email 0 2 1
ONCE thumbnails 0 3 3
EVERY report 1 4 2 1
ONCE email 3 1 1
ONCE bad 3 0 1
EVERY bad 3 0 1 1
RUN_UNTIL 1
STATUS
RUN_UNTIL 9
CANCEL report
RUN_UNTIL 10
STATUS
CANCEL report
ONCE late 3 1 2
ONCE cleanup 12 2 1
EVERY ping 12 3 1 2
RUN_UNTIL 15
STATUS
RUN_UNTIL 12
```

**expected output**

```
Scheduler ready: 2 workers
backup: once at t=0, 5s, priority 2
email: once at t=0, 2s, priority 1
thumbnails: once at t=0, 3s, priority 3
report: every 4s from t=1, 2s, priority 1
Task email already exists
Duration must be at least 1
Interval must be at least 1
t=0: email started on W1
t=0: backup started on W2
t=1
W1: email until t=2
W2: backup until t=5
Waiting: report, thumbnails
t=2: email finished on W1
t=2: report started on W1
t=4: report finished on W1
t=4: thumbnails started on W1
t=5: backup finished on W2
t=7: thumbnails finished on W1
t=8: report started on W1
Cancelled report (the current run will finish)
t=10: report finished on W1
t=10
W1: idle
W2: idle
Unknown task report
late: once at t=3, 1s, priority 2
cleanup: once at t=12, 2s, priority 1
ping: every 3s from t=12, 1s, priority 2
t=11: late started on W1
t=12: late finished on W1
t=12: cleanup started on W1
t=12: ping started on W2
t=13: ping finished on W2
t=14: cleanup finished on W1
t=15
W1: idle
W2: idle
Next: ping at t=16
Time cannot go back
```

## Part C — pressure questions

| | |
|---|---|
| 1 | **Real threads.** "Turn this into a real scheduler, with worker threads and the real clock." Sketch it. |
| 2 | **Failure.** "A task throws an exception." What happens to the worker and to the next runs? |
| 3 | **Starvation.** "Priority 3 tasks never run on a busy day." |
| 4 | **Scale.** "Run it on five machines, and every task must run exactly once." |
| 5 | **Defend it.** "Why two structures? Keep one sorted list of tasks." |

# Problem 4 — Food Delivery

**Heavy** · 100 minutes (25 design + 75 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Design Swiggy or Zomato. Restaurants have menus. A customer orders from one restaurant. The restaurant accepts or rejects the order. We assign the nearest free delivery partner, who picks it up and delivers it. Customers get updates and can cancel before pickup. When no partner is free, orders wait."

### After your questions, the requirements are

| | |
|---|---|
| **R1** | Restaurants, customers and partners have a location on a grid of streets; distance = \|x1 − x2\| + \|y1 − y2\| km. Restaurants have a menu of items with prices. |
| **R2** | An order has a customer, one restaurant, items with quantities (at least 1), and a delivery fee of Rs 20 + Rs 5 per km from restaurant to customer. Order IDs are O1, O2, .... Refused, in this order, if: an item is badly written (not `name:qty`), the customer is unknown, the restaurant is unknown, or, item by item, it isn't on the menu or its quantity is below 1. |
| **R3** | Life cycle: PLACED → ACCEPTED or REJECTED (by the restaurant) → PICKED_UP → DELIVERED. The customer can cancel while PLACED or ACCEPTED. Any other move is refused. |
| **R4** | On acceptance, the order gets the free partner closest to the restaurant (a tie goes to the name in alphabetical order). If none is free, it waits in a queue. An order can't be picked up without a partner. |
| **R5** | On delivery, the partner is now at the customer's location and is free again. On cancellation, a partner assigned to the order is freed. A freed partner immediately takes the oldest waiting order. |
| **R6** | Rejected and cancelled orders are refunded in full. The customer is notified when the order is picked up, delivered, rejected or cancelled; notifications print after the command's result. |
| | |
| **NF1** | The assignment rule (nearest, least busy, highest rated) and the fee rule (surge pricing) will change; neither should touch order handling. |
| **NF2** | Notifications (app push, SMS) must not be tangled into order handling. |
| **NF3** | Many restaurants accept orders at the same moment: one partner never gets two orders (RUSH). |

**Out of scope:** payments, ratings, live GPS tracking, orders from several restaurants, partners carrying two orders.

### Commands and exact output

| Command | Output |
|---|---|
| `RESTAURANT / CUSTOMER / PARTNER name x y` | `Restaurant Vaishali at (0,0)` · `Customer Asha at (3,4)` · `Partner Kiran at (1,1)` |
| `ITEM restaurant item price` | `Vaishali menu: Dosa Rs 120` |
| `ORDER customer restaurant item:qty …` | `O1 placed: Asha from Vaishali, items Rs 280 + delivery Rs 55 = Rs 335` |
| `ACCEPT / REJECT order` | `O1 accepted, partner Kiran is 2 km away` · `O3 accepted, waiting for a partner` · `O4 rejected, refund Rs 335` |
| `PICKUP / DELIVER order` | `O1 picked up by Kiran` · `O1 delivered by Kiran`, then `O3 assigned to Kiran, 7 km away` if a waiting order got the partner |
| `CANCEL order` | `O2 cancelled, refund Rs 530` (plus an assignment line, as above) |
| `TRACK order` | `O3: ACCEPTED, partner Kiran` or `O2: CANCELLED` |
| `RUSH customer restaurant item n` | places n orders of one item, then accepts them all at the same moment: `Rush: 1 got a partner, 9 waiting` |
| `(notification)` | `-> Asha: order O1 picked up` (also `delivered`, `rejected`, `cancelled`) |

| Error | Output |
|---|---|
| `unknown customer / restaurant / order` | `Unknown customer Neha` · `Unknown restaurant X` · `Unknown order O9` |
| `item not on the menu / bad quantity / bad item text` | `Vaishali has no Pizza` · `Quantity must be at least 1` · `Bad item Dosa` |
| `a move the life cycle doesn't allow` | `Cannot pick up O1: it is PLACED` (also `accept`, `reject`, `deliver`, `cancel`) |
| `pickup before a partner is assigned` | `O3 has no partner yet` |

### Part A — design

| | |
|---|---|
| 1 | The actors and entities, and who owns what. Draw the order's life cycle. |
| 2 | The life cycle: State pattern, or an enum with a list of allowed moves? Decide, and say why. |
| 3 | NF1 and NF2: which patterns, and where? |
| 4 | NF3: what must be one indivisible step when assigning a partner? What happens to the waiting queue when a partner is freed? |

## Part B — code

**input.txt**

```
RESTAURANT Vaishali 0 0
RESTAURANT Wadeshwar 10 10
ITEM Vaishali Dosa 120
ITEM Vaishali Coffee 40
ITEM Wadeshwar Thali 250
CUSTOMER Asha 3 4
CUSTOMER Ravi 9 9
PARTNER Kiran 1 1
PARTNER Meera 8 8
ORDER Asha Vaishali Dosa:2 Coffee:1
ORDER Ravi Wadeshwar Thali:2
ORDER Asha Vaishali Pizza:1
ORDER Neha Vaishali Dosa:1
ORDER Asha Vaishali Dosa:0
PICKUP O1
ACCEPT O1
ACCEPT O2
ORDER Ravi Vaishali Coffee:3
ACCEPT O3
PICKUP O3
REJECT O3
PICKUP O1
DELIVER O1
CANCEL O1
TRACK O3
ORDER Asha Wadeshwar Thali:1
REJECT O4
CANCEL O2
TRACK O2
TRACK O9
RUSH Asha Vaishali Coffee 10
```

**expected output**

```
Restaurant Vaishali at (0,0)
Restaurant Wadeshwar at (10,10)
Vaishali menu: Dosa Rs 120
Vaishali menu: Coffee Rs 40
Wadeshwar menu: Thali Rs 250
Customer Asha at (3,4)
Customer Ravi at (9,9)
Partner Kiran at (1,1)
Partner Meera at (8,8)
O1 placed: Asha from Vaishali, items Rs 280 + delivery Rs 55 = Rs 335
O2 placed: Ravi from Wadeshwar, items Rs 500 + delivery Rs 30 = Rs 530
Vaishali has no Pizza
Unknown customer Neha
Quantity must be at least 1
Cannot pick up O1: it is PLACED
O1 accepted, partner Kiran is 2 km away
O2 accepted, partner Meera is 4 km away
O3 placed: Ravi from Vaishali, items Rs 120 + delivery Rs 110 = Rs 230
O3 accepted, waiting for a partner
O3 has no partner yet
Cannot reject O3: it is ACCEPTED
O1 picked up by Kiran
-> Asha: order O1 picked up
O1 delivered by Kiran
O3 assigned to Kiran, 7 km away
-> Asha: order O1 delivered
Cannot cancel O1: it is DELIVERED
O3: ACCEPTED, partner Kiran
O4 placed: Asha from Wadeshwar, items Rs 250 + delivery Rs 85 = Rs 335
O4 rejected, refund Rs 335
-> Asha: order O4 rejected
O2 cancelled, refund Rs 530
-> Ravi: order O2 cancelled
O2: CANCELLED
Unknown order O9
Rush: 1 got a partner, 9 waiting
```

## Part C — pressure questions

| | |
|---|---|
| 1 | **Change.** "Surge pricing: when more than 5 orders are waiting, the delivery fee doubles." |
| 2 | **Change.** "A partner can carry two orders if both restaurants are close." |
| 3 | **Failure.** "A partner's phone goes offline after accepting an order." |
| 4 | **Scale.** "Fifty cities, a million orders a day, partners sending GPS every few seconds." |
| 5 | **Defend it.** "Use the State pattern for the order, like you did for the vending machine." |

# Hints

Read one level at a time, and only after 15 minutes of being stuck. Each level gives away more than the last.

## Problem 1 — Logging Framework

**Hint 1 (a nudge).** Two things vary independently: *where* a line goes and *how* it looks. A third thing is a lookup: which level applies to this logger.

**Hint 2 (the shape).** `Level` enum (declared in order) · `LogEvent` record · `Formatter` interface with `PatternFormatter` · abstract `Appender` with `ConsoleAppender` and `BufferedAppender` · `Logger` (a name and a link to the manager) · `LogManager` (levels by name, loggers by name, appenders, counters).

**Hint 3 (the key decision).** Effective level: look up the name; if missing, cut at the last dot and look again; at no dots, use root. Then compare with `compareTo`: drop if below, else send to each appender whose minimum it meets.

## Problem 2 — Hotel Booking

**Hint 1.** It's the meeting-room problem with dates instead of minutes, and the guest chooses a type instead of a room.

**Hint 2.** `RoomType` enum with a base price · `Room` (number, type, a `TreeMap` from check-in date to booking) · `Booking` · `PricingStrategy` and `RefundPolicy` interfaces · `Hotel` as the front door. `LocalDate` gives you the day of the week and the days between two dates.

**Hint 3.** Free check: `lowerEntry(checkOut)` and whether its check-out is after your check-in. Booking: walk the rooms of that type in number order; for each, `synchronized (room)` { if free, create the booking and add it }. Take the booking number only inside, after the check.

## Problem 3 — Task Scheduler

**Hint 1.** "When is it due?" and "who goes first once it's due?" are different questions with different orders.

**Hint 2.** `Task` (name, priority, duration, interval, next due, a creation number) · `Worker` (current task, busy until) · a `TreeSet` ordered by due time for the future · a `PriorityQueue` ordered by priority, due time and creation for the waiting tasks.

**Hint 3.** One second: finish runs ending now (and re-add repeating tasks to the future set with now + interval), then move everything due from the future set to the waiting queue, then give idle workers the head of the queue. Cancel = a flag plus removal from both structures.

## Problem 4 — Food Delivery

**Hint 1.** Separate the life cycle (what moves are allowed), the dispatch (which partner, and the waiting queue) and the side effects (fees, notifications). Each one changes for a different reason.

**Hint 2.** `Location` record with `distanceTo` · `Restaurant`, `Customer`, `Partner`, `Order` · `OrderStatus` enum with `canMoveTo(next)` · `DeliveryFee` and `AssignmentStrategy` interfaces · `OrderListener` with a `CustomerNotifier` · `FoodService` as the front door.

**Hint 3.** One private `move(id, next, verb)` does every status change: check `canMoveTo`, set the status, tell the listeners. Assigning = choose among free partners and mark busy, under the one lock. Freeing a partner always ends with "give it to the oldest waiting order".

# Solution 1 — Logging Framework

## Questions a strong candidate asks first

| Question | Answer assumed here |
|---|---|
| Which levels? Can each package have its own? | DEBUG to ERROR; yes, inherited from the parent |
| Which destinations? Different formats per destination? | Console and a buffered "file"; yes, a pattern each |
| Can a destination have its own minimum level? | Yes |
| Many threads? Must logging be fast? | Yes and yes (Part C 1) |
| Configuration from a file? | Not now: by commands |

## Part A — model answer

```
LogManager 1 ———> * Logger (one per name)      LogManager ———> Map<String, Level> levels ("root" = INFO)
LogManager 1 ———> * «abstract» Appender  <———  ConsoleAppender, BufferedAppender
Appender ———> «interface» Formatter  <- - -  PatternFormatter                LogEvent: record
```

**1. Who owns what.** The **manager** owns the level settings, the loggers and the appenders, so it decides whether a message passes. A **Logger** is just a name plus a link to the manager: what application code holds. An **appender** decides where a line goes, and its **formatter** decides how it looks. A **LogEvent** record carries one message to every appender.

**2. Effective level: walk up the dots.** Look up `com.app.payment`; if it has no level, try `com.app`, then `com`, then `root`. It lives in the manager, which holds the settings. Setting `com.app` to DEBUG changes every logger under it at once, and a child's own setting wins over its parent's.

**3. Two Strategies, chosen separately.** Destinations vary (console, file, Kafka) and formats vary (pattern, JSON), independently. Appender is one hierarchy, Formatter another, and each appender is given a formatter: any destination with any format, and NF1 is one new class either way. The appenders together work like Observer: the manager tells all of them about each event.

**4. Chain of Responsibility for levels? No.** Levels are an ordered enum, so "is WARN at least INFO?" is one comparison. A chain of level handlers adds objects and calls to do the same thing. CoR fits when each handler may *stop* the request, like approval limits; here every appender gets the event independently.

**In the interview: write this first.** (1) `Level`, `LogEvent`, `Formatter` with the pattern: 10 minutes. (2) `Appender` and the console appender. (3) The manager: levels, the walk up, `log`. (4) The buffered appender. (5) Stats. The domain code here is about 145 lines.

## Part B — the code

```
import java.util.*;

// Declared from least to most severe, so compareTo() compares severity.
enum Level {
    DEBUG, INFO, WARN, ERROR;

    static Level parse(String s) {
        for (Level l : values()) if (l.name().equals(s)) return l;
        throw new IllegalArgumentException("Unknown level " + s);
    }
}

record LogEvent(String time, Level level, String logger, String message) {}

interface Formatter {
    String format(LogEvent e);
}

class PatternFormatter implements Formatter {      // %t time, %l level, %c logger, %m message
    private final String pattern;

    PatternFormatter(String pattern) { this.pattern = pattern; }

    public String format(LogEvent e) {
        StringBuilder out = new StringBuilder();
        for (int i = 0; i < pattern.length(); i++) {
            char c = pattern.charAt(i);
            if (c == '%' && i + 1 < pattern.length()) {
                char k = pattern.charAt(++i);
                switch (k) {
                    case 't' -> out.append(e.time());
                    case 'l' -> out.append(e.level());
                    case 'c' -> out.append(e.logger());
                    case 'm' -> out.append(e.message());
                    default -> out.append('%').append(k);
                }
            } else {
                out.append(c);
            }
        }
        return out.toString();
    }
}

abstract class Appender {                          // a destination: console, file, network...
    final String name;
```

```
    final Level minLevel;
    final Formatter formatter;

    Appender(String name, Level minLevel, Formatter formatter) {
        this.name = name; this.minLevel = minLevel; this.formatter = formatter;
    }

    abstract void append(LogEvent e);
    int flush() { return 0; }                       // returns how many lines it wrote

    protected void write(String line) { System.out.println(name + "| " + line); }
}

class ConsoleAppender extends Appender {
    ConsoleAppender(String name, Level min, Formatter f) { super(name, min, f); }

    void append(LogEvent e) { write(formatter.format(e)); }
}

class BufferedAppender extends Appender {          // like a file: writes in batches
    private final int size;
    private final List<String> buffer = new ArrayList<>();

    BufferedAppender(String name, Level min, Formatter f, int size) {
        super(name, min, f);
        this.size = size;
    }

    void append(LogEvent e) {
        buffer.add(formatter.format(e));
        if (buffer.size() == size) flush();
    }

    @Override int flush() {
        int n = buffer.size();
        buffer.forEach(this::write);
        buffer.clear();
        return n;
    }
}

class Logger {                                     // what application code holds
    final String name;
    private final LogManager manager;

    Logger(String name, LogManager manager) { this.name = name; this.manager = manager; }

    void debug(String msg) { manager.log(name, Level.DEBUG, msg); }
    void info(String msg)  { manager.log(name, Level.INFO, msg); }
    void warn(String msg)  { manager.log(name, Level.WARN, msg); }
    void error(String msg) { manager.log(name, Level.ERROR, msg); }
    void log(Level level, String msg) { manager.log(name, level, msg); }
}

class LogManager {
    private final Map<String, Level> levels = new HashMap<>();
    private final Map<String, Logger> loggers = new HashMap<>();
    private final List<Appender> appenders = new ArrayList<>();
    private String time = "00:00:00";
    private int logged, filtered;

    LogManager() { levels.put("root", Level.INFO); }

    synchronized Logger getLogger(String name) {   // one Logger object per name
        return loggers.computeIfAbsent(name, n -> new Logger(n, this));
    }

    synchronized void setLevel(String logger, Level level) { levels.put(logger, level); }
    synchronized void addAppender(Appender a)               { appenders.add(a); }
    synchronized void setTime(String t)                     { time = t; }

    // A logger without its own level uses its nearest ancestor's:
    // com.app.pay -> com.app -> com -> root.
    synchronized Level effectiveLevel(String logger) {
        String name = logger;
        while (true) {
            Level l = levels.get(name);
            if (l != null) return l;
            int dot = name.lastIndexOf('.');
            if (dot < 0) return levels.get("root");
            name = name.substring(0, dot);
        }
    }

    synchronized void log(String logger, Level level, String message) {
        if (level.compareTo(effectiveLevel(logger)) < 0) { filtered++; return; }
        logged++;
        LogEvent e = new LogEvent(time, level, logger, message);
        for (Appender a : appenders)
            if (level.compareTo(a.minLevel) >= 0) a.append(e);
    }

    synchronized int flushAll() {
        int lines = 0;
        for (Appender a : appenders) lines += a.flush();
        return lines;
    }

    synchronized String stats() { return "Logged " + logged + ", filtered " + filtered; }
}
```

```
public class Main {
    public static void main(String[] args) {
        LogManager manager = new LogManager();
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "TIME" -> { manager.setTime(p[1]); System.out.println("Time " + p[1]); }
                    case "LEVEL" -> {
                        manager.setLevel(p[1], Level.parse(p[2]));
                        System.out.println("Level of " + p[1] + ": " + p[2]);
                    }
                    case "CONSOLE" -> {
                        Formatter f = new PatternFormatter(rest(line, 3));
                        manager.addAppender(new ConsoleAppender(p[1], Level.parse(p[2]), f));
                        System.out.println("Appender " + p[1] + ": console, " + p[2]
                                + " and above");
                    }
                    case "BUFFER" -> {
                        Formatter f = new PatternFormatter(rest(line, 4));
                        manager.addAppender(new BufferedAppender(p[1], Level.parse(p[2]), f,
                                Integer.parseInt(p[3])));
                        System.out.println("Appender " + p[1] + ": buffer of " + p[3] + ", " + p[2]
                                + " and above");
                    }
                    case "LOG" -> manager.getLogger(p[1]).log(Level.parse(p[2]), rest(line, 3));
                    case "FLUSH" -> {
                        if (manager.flushAll() == 0) System.out.println("Nothing to flush");
                    }
                    case "STATS" -> System.out.println(manager.stats());
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    // Everything after the first n words of the line, spaces kept.
    static String rest(String line, int n) {
        String s = line;
        for (int i = 0; i < n; i++) s = s.substring(s.indexOf(' ') + 1).stripLeading();
        return s;
    }
}
```

**How to read it:** `log` does exactly R4: compare with the effective level, then hand the same event to each appender that wants it. The pattern formatter walks the pattern once, character by character, so a message containing "%l" is never treated as a pattern.

## Part C — model answers

**1. Writing is slowing the API.** Make writing asynchronous: `log` only puts the event on a bounded queue, and one background thread takes events off and writes them in batches. The request thread returns at once. Decide what happens when the queue is full: block, or drop DEBUG lines first. Flush the queue on shutdown. Log4j 2 and Logback both offer async appenders built this way.

**2. Appenders per logger.** Each logger keeps its own list of appenders and an "additive" flag. An event goes to its logger's appenders, then walks up the dots to each ancestor's, stopping at a logger whose flag is false. That's the same walk as the level lookup.

**3. The disk is full.** Logging must never crash the application. Catch the exception inside the appender, report it once to standard error, and keep going: drop lines, or fall back to the console. Logging is an announcement, not a required step.

**4. Strings built for nothing.** Check before building ( `if (log.isDebugEnabled())` ), or pass the parts and let the logger build the string only if it will be written: `log.debug("user {}", user)` , or a `Supplier<String>` .

**5. "Make it a Singleton."** "You'd be right that libraries expose a static `LoggerFactory.getLogger` , because every class needs a logger and passing one everywhere is noisy. That's a convenience at the edge. Inside, I keep `LogManager` an ordinary class created once and passed in, so tests can make a fresh one with their own appenders. The static method can just hand out that one instance."

**Common mistakes**

- A level comparison written as a chain of `if` s on strings instead of an ordered enum.
- Only checking the logger's own level, without walking up to its parents.
- Formatting with `String.replace` , so a message that contains "%m" gets mangled.
- Logging code that knows about files or the console.
- An appender failure that throws into the application.

## Score yourself (1 point each)

| | | | |
|---|---|---|---|
| ☐ | Levels are an ordered enum | ☐ | One Logger object per name |
| ☐ | Effective level walks up to root | ☐ | Filtered messages reach no appender |
| ☐ | Each appender has its own minimum level | ☐ | Appender and Formatter are separate |
| ☐ | The buffer writes when full and on FLUSH | ☐ | Your output matches exactly |
| ☐ | Part C 1: async writing with a queue | ☐ | Part A 4: you can say why not CoR |

# Solution 2 — Hotel Booking

## Questions a strong candidate asks first

| Question | Answer assumed here |
|---|---|
| Does the guest pick a room, or a type? | A type; we assign the room |
| Can someone check in on the day another checks out? | Yes: a stay covers nights, check-in to check-out − 1 |
| How are prices worked out? | Per night; Friday and Saturday nights +20% |
| What's the refund rule? | By days before check-in: 100%, 50% or 0% |
| Several hotels? Payment? | One hotel; no payment |

## Part A — model answer

```
Hotel 1 ◆——— * Room 1 ◆——— * Booking         Room: number, RoomType, TreeMap<LocalDate, Booking>
Hotel ———> «interface» PricingStrategy  ◁- - -  WeekendPricing
Hotel ———> «interface» RefundPolicy     ◁- - -  StandardRefund        RoomType: enum with base price
```

**1. Who owns what.** A **room** owns its calendar and its lock. A **room type** owns its base price. The **pricing strategy** owns "what does this night cost?", and the **refund policy** owns "how much back, this many days before?". The **hotel** coordinates: search, book, cancel.

**2. The free check.** As with meeting rooms: a room's bookings never overlap, so only the last booking starting before your check-out can clash. `lowerEntry(checkOut)`, then: is its check-out after your check-in? Dates instead of minutes; same idea, O(log n).

**3. Pricing and refunds are Strategies (NF1).** Each is a one-method interface given to the hotel. Diwali pricing or a non-refundable rate is a new class (or a decorator around the existing one), and booking code doesn't change.

**4. One step per room.** For each room of the type, in number order: `synchronized (room) { is it free? then create and add the booking }`. If a room is taken by someone faster, move on to the next. Rooms never wait for each other. Tested: with that lock removed, RUSH sold one room twice ("3 booked" with only 2 rooms free) in **10 runs out of 60**.

**In the interview: write this first.** (1) `RoomType`, `Room` with `isFree`, `Booking`: 15 minutes. (2) `Hotel.book` with the per-room lock. (3) Pricing. (4) Cancel and refunds. (5) Search, then RUSH. The domain code here is about 145 lines.

## Part B — the code

```
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.time.temporal.ChronoUnit;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.IntPredicate;

enum RoomType {
    STANDARD(2000), DELUXE(3500), SUITE(6000);

    final int basePrice;                                  // per night
    RoomType(int basePrice) { this.basePrice = basePrice; }

    static RoomType parse(String s) {
        for (RoomType t : values()) if (t.name().equals(s)) return t;
        throw new IllegalArgumentException("Unknown room type " + s);
    }
}

interface PricingStrategy {
    int priceFor(RoomType type, LocalDate night);
}

// Friday and Saturday nights cost 20% more
class WeekendPricing implements PricingStrategy {
    public int priceFor(RoomType type, LocalDate night) {
        DayOfWeek d = night.getDayOfWeek();
        boolean weekend = d == DayOfWeek.FRIDAY || d == DayOfWeek.SATURDAY;
        return weekend ? type.basePrice * 120 / 100 : type.basePrice;
    }
}

interface RefundPolicy {
    int percent(long daysBeforeCheckIn);
}

class StandardRefund implements RefundPolicy {
    public int percent(long days) { return days >= 7 ? 100 : days >= 2 ? 50 : 0; }
}

class Booking {
    final String id, guest;
    final Room room;
    final LocalDate checkIn, checkOut;                    // nights: checkIn .. checkOut - 1
    final int amount;
    boolean cancelled;                                    // changed under the room's lock
```

```
    Booking(String id, String guest, Room room, LocalDate in, LocalDate out, int amount) {
        this.id = id; this.guest = guest; this.room = room;
        this.checkIn = in; this.checkOut = out; this.amount = amount;
    }

    long nights() { return ChronoUnit.DAYS.between(checkIn, checkOut); }
}

// Its calendar is guarded by its own lock
class Room {
    final int number;
    final RoomType type;
    private final TreeMap<LocalDate, Booking> byCheckIn = new TreeMap<>();

    Room(int number, RoomType type) { this.number = number; this.type = type; }

    // Bookings in one room never overlap, so only the last one starting before `out` can clash.
    synchronized boolean isFree(LocalDate in, LocalDate out) {
        Map.Entry<LocalDate, Booking> before = byCheckIn.lowerEntry(out);
        return before == null || !before.getValue().checkOut.isAfter(in);
    }

    synchronized void add(Booking b)    { byCheckIn.put(b.checkIn, b); }
    synchronized void remove(Booking b) { byCheckIn.remove(b.checkIn); }
}

class Hotel {
    private final Map<Integer, Room> rooms = new TreeMap<>();          // by room number
    private final Map<String, Booking> bookings = new ConcurrentHashMap<>();
    private final AtomicInteger ids = new AtomicInteger();
    private final PricingStrategy pricing;
    private final RefundPolicy refunds;

    Hotel(PricingStrategy pricing, RefundPolicy refunds) {
        this.pricing = pricing;
        this.refunds = refunds;
    }

    void addRoom(int number, RoomType type) {
        if (rooms.containsKey(number))
            throw new IllegalArgumentException("Room " + number + " already exists");
        rooms.put(number, new Room(number, type));
    }

    int price(RoomType type, LocalDate in, LocalDate out) {
        int total = 0;
        for (LocalDate night = in; night.isBefore(out); night = night.plusDays(1))
            total += pricing.priceFor(type, night);
        return total;
    }

    long freeCount(RoomType type, LocalDate in, LocalDate out) {
        checkDates(in, out);
        return rooms.values().stream().filter(r -> r.type == type && r.isFree(in, out)).count();
    }

    // The lowest-numbered room of that type that is free for every night.
    Booking book(String guest, RoomType type, LocalDate in, LocalDate out) {
        checkDates(in, out);
        for (Room room : rooms.values()) {
            if (room.type != type) continue;
            synchronized (room) {                                      // check and add: one step
                if (!room.isFree(in, out)) continue;
                Booking b = new Booking("B" + ids.incrementAndGet(), guest, room, in, out,
                        price(type, in, out));
                room.add(b);
                bookings.put(b.id, b);
                return b;
            }
        }
        throw new IllegalStateException("No " + type + " room free for " + in + " to " + out);
    }

    // Returns the refund in rupees.
    int cancel(String id, LocalDate today) {
        Booking b = bookings.get(id);
        if (b == null) throw new IllegalArgumentException("Unknown booking " + id);
        synchronized (b.room) {
            if (b.cancelled) throw new IllegalStateException(id + " is already cancelled");
            long days = ChronoUnit.DAYS.between(today, b.checkIn);
            if (days <= 0) throw new IllegalStateException("Cannot cancel on or after check-in");
            b.cancelled = true;
            b.room.remove(b);
            return b.amount * refunds.percent(days) / 100;
        }
    }

    Booking booking(String id) { return bookings.get(id); }

    private void checkDates(LocalDate in, LocalDate out) {
        if (!out.isAfter(in))
            throw new IllegalArgumentException("Check-out must be after check-in");
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        Hotel hotel = new Hotel(new WeekendPricing(), new StandardRefund());
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
```

```
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "ROOM" -> {
                        hotel.addRoom(Integer.parseInt(p[1]), RoomType.parse(p[2]));
                        System.out.println("Room " + p[1] + " (" + p[2] + ") added");
                    }
                    case "SEARCH" -> {
                        LocalDate from = date(p[1]), to = date(p[2]);
                        for (RoomType t : RoomType.values()) {
                            long free = hotel.freeCount(t, from, to);
                            System.out.println(free == 0 ? t + ": none free"
                                    : t + ": " + free + " free, Rs " + hotel.price(t, from, to));
                        }
                    }
                    case "BOOK" -> {
                        Booking b = hotel.book(p[1], RoomType.parse(p[2]), date(p[3]), date(p[4]));
                        System.out.println(b.id + ": " + b.guest + ", room " + b.room.number + " ("
                                + b.room.type + "), " + b.checkIn + " to " + b.checkOut + ", "
                                + b.nights() + (b.nights() == 1 ? " night" : " nights")
                                + ", Rs " + b.amount);
                    }
                    case "CANCEL" -> {
                        LocalDate today = date(p[2]);
                        int refund = hotel.cancel(p[1], today);
                        Booking b = hotel.booking(p[1]);
                        long days = ChronoUnit.DAYS.between(today, b.checkIn);
                        String when = days + (days == 1 ? " day" : " days");
                        System.out.println(p[1] + " cancelled " + when
                                + " before check-in, refund Rs "
                                + refund + " (" + refund * 100 / b.amount + "%)");
                    }
                    // N guests book the same type and dates at once
                    case "RUSH" -> {
                        RoomType type = RoomType.parse(p[1]);
                        LocalDate from = date(p[2]), to = date(p[3]);
                        int n = Integer.parseInt(p[4]);
                        int booked = rush(n, i -> {
                            try {
                                hotel.book("guest" + i, type, from, to);
                                return true;
                            } catch (IllegalStateException full) {
                                return false;
                            }
                        });
                        System.out.println("Rush: " + booked + " booked, " + (n - booked)
                                + " refused");
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    static LocalDate date(String s) {
        try {
            return LocalDate.parse(s);                          // yyyy-mm-dd
        } catch (DateTimeParseException e) {
            throw new IllegalArgumentException("Invalid date " + s);
        }
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

**How to read it:** `book` walks rooms in number order because the map is a `TreeMap`. `price` adds one night at a time and asks the strategy for each, so a festival rule only has to answer for one night. The refund is `amount * percent / 100` in whole rupees.

## Part C — model answers

**1. Diwali week and long stays.** Diwali is a per-night rule: a `FestivalPricing` decorator wraps the weekend pricing and adds 50% on nights inside its dates. The long-stay discount depends on the whole stay, not one night, so it applies to the total: a second step after summing (a `StayDiscount` interface). Say this split out loud; it's the design point.

**2. A 10-minute hold.** Same as BookMyShow: the booking starts HELD with an expiry time, a scheduled job releases unpaid holds, and payment moves it to CONFIRMED. The difference: a hotel hold covers many nights of one room, and the free check treats HELD bookings as taken.

**3. Many hotels, a database, several servers.** Bookings become rows. Inside one transaction, lock the room's row ( `SELECT … FOR UPDATE` ), check for overlap with `check_in < :out AND check_out > :in` , then insert; or use a database constraint that refuses overlapping ranges. Many systems also keep an inventory table, "free rooms of this type per night", and decrement each night's count with a conditional update.

**4. Overbooking.** Counting rooms per type per night is what makes overbooking possible: allow the night's booked count to go up to 105% of the rooms, and assign actual room numbers at check-in instead of at booking. Today's design assigns a room at booking time, which can't oversell. That's a real trade-off to name.

**5. "Let the guest pick the room."** "You'd be right for a small guesthouse where guests ask for 'the room with the balcony'. For a hotel, guests care about the type, and assigning lets us fill rooms tightly: if guests picked, they'd leave one-night gaps that no one can book. We can still offer 'choose a room' as an extra."

**Common mistakes**

- Treating check-out day as a booked night, so back-to-back stays are refused.
- Pricing with `double` , or applying the weekend rule to check-in day only.
- Picking a free room in one step and booking it in another.
- Counting days before check-in the wrong way round, or allowing a cancel after check-in.
- Putting refund percentages inside `cancel()` with `if` s.

## Score yourself (1 point each)

| | | | |
|---|---|---|---|
| ☐ | A stay is the nights check-in to check-out − 1 | ☐ | Each room owns a sorted calendar |
| ☐ | The free check is one lookup | ☐ | The lowest free room of the type, per-room lock |
| ☐ | Pricing is a strategy, night by night | ☐ | Refunds are a strategy by days before |
| ☐ | Errors come out in the stated order | ☐ | Your output matches exactly, RUSH included |
| ☐ | Part C 1: per-night vs whole-stay rules | ☐ | Part C 3: a safe database booking |

# Solution 3 — Task Scheduler

## Questions a strong candidate asks first

| Question | Answer assumed here |
|---|---|
| How many workers? Can it change? | Fixed at start |
| Repeats: by the clock, or after each run finishes? | After each run (fixed delay) |
| What decides the order when several are due? | Priority, then due time, then age |
| What does cancel do to a running task? | Lets the run finish; nothing more |
| Real threads, or a simulation for the test? | Simulated clock here; threads in Part C 1 |

## Part A — model answer

```
Scheduler ———> * Worker (current task, busy until)        Scheduler ———> Map<String, Task>
Scheduler ———> TreeSet<Task> timeline     not due yet: by next due time, then creation
Scheduler ———> PriorityQueue<Task> ready  due, waiting: by priority, then due time, then creation
```

**1. Two groups, two orders.** Future tasks only ever need "which is due next?": a `TreeSet` by due time. Waiting tasks only need "which goes first?": a `PriorityQueue` by priority. One structure can't serve both orders cheaply. Java's `ScheduledThreadPoolExecutor` and cron-like systems split them the same way.

**2. The three steps, in order.** Finish first, so a worker that frees up at t can take a task at t. Then move what's due, so tasks due at t can be picked at t. Then assign. In any other order, workers sit idle for a second, or a task waits a second it shouldn't.

**3. Fixed delay, not fixed rate.** With fixed rate, a run that takes longer than its interval lets the next run become due while it's still going: runs pile up or overlap. Fixed delay (next due = finish + interval) means one run at a time per task. Java offers both: `scheduleAtFixedRate` and `scheduleWithFixedDelay`.

**4. NF1: the order is one Comparator.** "Priority, then due time, then creation" is the queue's comparator. Ageing is a different comparator, for example one that subtracts waiting time from priority (Part C 3). The scheduling loop doesn't change.

**In the interview: write this first.** (1) `Task` and `Worker`. (2) The two structures and `schedule`. (3) `runUntil` with its three steps: 20 minutes. (4) Cancel. (5) Status. The domain code here is about 110 lines.

## Part B — the code

```
import java.util.*;

class Task {
    final String name;
    // Priority 1 is the most urgent; interval 0 = once
    final int priority, duration, interval;
    // Order of creation, the last tie-break
    final long seq;
    long nextDue;
    boolean cancelled;

    Task(String name, int priority, int duration, int interval, long firstDue, long seq) {
        this.name = name; this.priority = priority; this.duration = duration;
        this.interval = interval; this.nextDue = firstDue; this.seq = seq;
    }
}

class Worker {
    final String id;
    Task current;                                          // null when idle
    long busyUntil;

    Worker(String id) { this.id = id; }
}

class Scheduler {
    private final List<Worker> workers = new ArrayList<>();
    private final Map<String, Task> tasks = new HashMap<>();
    // Not yet due, soonest first.
    private final TreeSet<Task> timeline = new TreeSet<>(
            Comparator.comparingLong((Task t) -> t.nextDue).thenComparingLong(t -> t.seq));
    // Due and waiting for a worker: most urgent first, then the one due earliest, then the oldest.
    private final PriorityQueue<Task> ready = new PriorityQueue<>(
            Comparator.comparingInt((Task t) -> t.priority).thenComparingLong(t -> t.nextDue)
                    .thenComparingLong(t -> t.seq));
    private long now = -1;                                 // the last second already processed
    private long seq;

    Scheduler(int workerCount) {
        for (int i = 1; i <= workerCount; i++) workers.add(new Worker("W" + i));
    }

    synchronized void schedule(String name, long firstDue, int interval, int duration,
                               int priority) {
        if (tasks.containsKey(name))
            throw new IllegalArgumentException("Task " + name + " already exists");
        if (duration < 1) throw new IllegalArgumentException("Duration must be at least 1");
        if (interval < 0) throw new IllegalArgumentException("Interval cannot be negative");
        Task t = new Task(name, priority, duration, interval, firstDue, seq++);
```

```
        tasks.put(name, t);
        timeline.add(t);
    }

    // Returns true if a run was in progress (it finishes, but nothing more is scheduled).
    synchronized boolean cancel(String name) {
        Task t = tasks.get(name);
        if (t == null || t.cancelled) throw new IllegalArgumentException("Unknown task " + name);
        t.cancelled = true;
        timeline.remove(t);
        ready.remove(t);
        return workers.stream().anyMatch(w -> w.current == t);
    }

    // Runs every second up to `until`, and returns what happened.
    synchronized List<String> runUntil(long until) {
        if (until < now) throw new IllegalArgumentException("Time cannot go back");
        List<String> events = new ArrayList<>();
        while (now < until) {
            now++;
            for (Worker w : workers) {                      // 1. finish what's done
                if (w.current != null && w.busyUntil == now) {
                    Task t = w.current;
                    events.add("t=" + now + ": " + t.name + " finished on " + w.id);
                    w.current = null;
                    if (t.interval > 0 && !t.cancelled) {    // repeat: next run after a fixed delay
                        t.nextDue = now + t.interval;
                        timeline.add(t);
                    }
                }
            }
            while (!timeline.isEmpty() && timeline.first().nextDue <= now)   // 2. what's due now
                ready.add(timeline.pollFirst());
            for (Worker w : workers) {                      // 3. hand work to idle workers
                if (w.current == null && !ready.isEmpty()) {
                    Task t = ready.poll();
                    w.current = t;
                    w.busyUntil = now + t.duration;
                    events.add("t=" + now + ": " + t.name + " started on " + w.id);
                }
            }
        }
        return events;
    }

    synchronized List<String> status() {
        List<String> out = new ArrayList<>();
        out.add("t=" + now);
        for (Worker w : workers)
            out.add(w.id + ": " + (w.current == null ? "idle"
                                                     : w.current.name + " until t=" + w.busyUntil));
        List<Task> waiting = new ArrayList<>(ready);
        waiting.sort(ready.comparator());
        if (!waiting.isEmpty())
            out.add("Waiting: " + String.join(", ", waiting.stream().map(t -> t.name).toList()));
        if (!timeline.isEmpty())
            out.add("Next: " + timeline.first().name + " at t=" + timeline.first().nextDue);
        return out;
    }
}

public class Main {
    public static void main(String[] args) {
        Scheduler scheduler = null;
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "WORKERS" -> {
                        scheduler = new Scheduler(Integer.parseInt(p[1]));
                        System.out.println("Scheduler ready: " + p[1] + " workers");
                    }
                    case "ONCE" -> {                        // ONCE name at duration priority
                        scheduler.schedule(p[1], Long.parseLong(p[2]), 0, Integer.parseInt(p[3]),
                                Integer.parseInt(p[4]));
                        System.out.println(p[1] + ": once at t=" + p[2] + ", " + p[3] + "s, priority "
                                + p[4]);
                    }
                    // EVERY name first interval duration priority
                    case "EVERY" -> {
                        int interval = Integer.parseInt(p[3]);
                        if (interval < 1)
                            throw new IllegalArgumentException("Interval must be at least 1");
                        scheduler.schedule(p[1], Long.parseLong(p[2]), interval,
                                Integer.parseInt(p[4]), Integer.parseInt(p[5]));
                        System.out.println(p[1] + ": every " + p[3] + "s from t=" + p[2] + ", "
                                + p[4] + "s, priority " + p[5]);
                    }
                    case "CANCEL" -> System.out.println("Cancelled " + p[1]
                            + (scheduler.cancel(p[1]) ? " (the current run will finish)" : ""));
                    case "RUN_UNTIL" ->
                            scheduler.runUntil(Long.parseLong(p[1])).forEach(System.out::println);
                    case "STATUS" -> scheduler.status().forEach(System.out::println);
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
```

```
        }
    }
}
```

**How to read it:** `runUntil` is R3, step by step. `status` copies the queue and sorts the copy, because iterating a `PriorityQueue` directly doesn't give its order. Cancel removes the task from both structures, and the flag stops a running repeating task from being added back.

## Part C — model answers

**1. Real threads.** One dispatcher thread waits until the soonest task is due (a `DelayQueue` does this: `take()` blocks until the head's delay runs out), then hands it to a `ThreadPoolExecutor` with W threads, whose queue is a `PriorityBlockingQueue` using the same comparator. A repeating task schedules its next run in a `finally` block when it finishes. In practice, `ScheduledThreadPoolExecutor` already does most of this.

**2. A task throws.** Catch everything around each run, so the worker thread survives, log it, and decide the policy: retry with a back-off (wait 1 s, 2 s, 4 s…), up to a limit, then mark the task failed and alert. A repeating task normally keeps its schedule. Note that Java's `scheduleAtFixedRate` quietly stops a task after it throws, a classic surprise worth mentioning.

**3. Starvation.** Ageing: the longer a task waits, the more urgent it becomes, for example effective priority = priority − seconds waited / 60. Because the order is one comparator, that's the change. Or reserve one worker for low-priority work.

**4. Five machines, exactly once.** Store tasks and their next due time in a database. A machine claims a due task with a conditional update ( `UPDATE task SET owner = me, lease_until = now + 30s WHERE id = ? AND (owner IS NULL OR lease_until < now)` ); one row changed means it's yours. The lease lets another machine take over if the owner dies. Strictly "exactly once" is hard, so make tasks safe to run twice (idempotent) and aim for at-least-once.

**5. "One sorted list."** "You'd be right for a handful of tasks: one list, scanned every second, is easy. But it has to be sorted two ways at once: by due time to find what's due, and by priority to pick among them. With one list, every second scans it all. Two structures give each question its own order, and it's how the Java scheduler is built."

**Common mistakes**

- Picking the next task by due time only, ignoring priority among the due ones.
- Assigning before finishing, so a worker freed at t stays idle until t + 1.
- Iterating a `PriorityQueue` and expecting sorted order.
- Rescheduling a cancelled repeating task when its last run finishes.
- Using `Thread.sleep` and real time in tests.

## Score yourself (1 point each)

| | | | |
|---|---|---|---|
| ☐ | Two structures: future and waiting | ☐ | The waiting order is one comparator |
| ☐ | Finish, then due, then assign, each second | ☐ | Repeats use a fixed delay after finishing |
| ☐ | Cancel lets a running run finish | ☐ | A task due in the past runs at once |
| ☐ | Status lists waiting tasks in order | ☐ | Your output matches exactly |
| ☐ | Part C 1: DelayQueue plus a thread pool | ☐ | Part C 4: claiming with a lease |

# Solution 4 — Food Delivery

## Questions a strong candidate asks first

| Question | Answer assumed here |
|---|---|
| One restaurant per order? One order per partner? | Yes and yes (Part C 2 asks about batching) |
| How is a partner chosen? When? | Nearest free one to the restaurant, when the restaurant accepts |
| What if no partner is free? | The order waits; the next freed partner takes the oldest |
| When can a customer cancel? Refund? | Before pickup; full refund |
| How is the delivery fee worked out? | Rs 20 + Rs 5 per km, restaurant to customer |
| Who gets notified, about what? | The customer: picked up, delivered, rejected, cancelled |

## Part A — model answer

```
FoodService ———> * Restaurant (menu)   * Customer   * Partner (location, current order)   * Order
FoodService ———> Deque<Order> waiting
FoodService ———> «interface» DeliveryFee         ◁- - -  DistanceFee
FoodService ———> «interface» AssignmentStrategy  ◁- - -  NearestPartner
FoodService ———> * «interface» OrderListener     ◁- - -  CustomerNotifier

PLACED ——accept——> ACCEPTED ——pick up——> PICKED_UP ——deliver——> DELIVERED
  ├——reject——> REJECTED
  └——cancel——> CANCELLED <——cancel—— ACCEPTED
```

**1. Who owns what.** A **restaurant** owns its menu; a **partner** owns its location and current order; an **order** owns its items, money and status; the **service** owns dispatch (free partners and the waiting queue), because that spans many orders and partners.

**2. An enum with allowed moves, not State.** Every move does the same thing: check it's allowed, change the status, notify. Only *which* moves are allowed differs, and that's data: `canMoveTo(next)`. One `move()` method uses it for all five commands. The State pattern earns its place when each state *behaves* differently, like the vending machine's coins and refunds; here it would be six classes that each say "allowed" or "not allowed".

**3. Patterns.** **Strategy** for the fee and for assignment (NF1): surge pricing or "highest rated partner" is a new class. **Observer** for notifications (NF2): the service tells its listeners after each status change, and the notifier collects messages the driver prints after the result. **Facade**: `FoodService` is the one front door.

**4. NF3: choose and mark busy in one step.** "Which partners are free?" + "pick one" + "mark them busy" must happen together, or two restaurants accepting at once both get Kiran. Here every service method is `synchronized`: one lock for dispatch, which is simple and fine at this scale. Freeing a partner (delivery or cancel) always ends with "give them to the oldest waiting order", inside the same lock. Tested: without the lock on `accept`, RUSH gave one partner two orders ("2 got a partner" with one free) in **16 runs out of 60**.

**In the interview: write this first.** (1) `Location`, the entities, `OrderStatus` with `canMoveTo`: 20 minutes. (2) `place` and `move`. (3) `accept` with nearest-partner assignment and the waiting queue. (4) Pick up, deliver and cancel, with freeing and reassignment. (5) The notifier, then RUSH. The domain code here is about 210 lines; if time is short, keep the fee and assignment as plain methods and say where the interfaces would go.

## Part B — the code

```
import java.util.*;
import java.util.concurrent.*;
import java.util.function.IntPredicate;

record Location(int x, int y) {
    // Km, along streets
    int distanceTo(Location o) { return Math.abs(x - o.x) + Math.abs(y - o.y); }
    @Override public String toString() { return "(" + x + "," + y + ")"; }
}

class Restaurant {
    final String name;
    final Location location;
    final Map<String, Integer> menu = new HashMap<>();          // item -> price

    Restaurant(String name, Location location) { this.name = name; this.location = location; }
}

record Customer(String name, Location location) {}

class Partner {
    final String name;
    Location location;
    Order current;                                              // null when free

    Partner(String name, Location location) { this.name = name; this.location = location; }
}

enum OrderStatus {
    PLACED, ACCEPTED, PICKED_UP, DELIVERED, REJECTED, CANCELLED;

    boolean canMoveTo(OrderStatus next) {
        return switch (this) {
            case PLACED -> next == ACCEPTED || next == REJECTED || next == CANCELLED;
            case ACCEPTED -> next == PICKED_UP || next == CANCELLED;
```

```
            case PICKED_UP -> next == DELIVERED;
            // DELIVERED, REJECTED, CANCELLED are final
            default -> false;
        };
    }
}

class Order {
    final String id;
    final Customer customer;
    final Restaurant restaurant;
    final int itemTotal, deliveryFee;
    OrderStatus status = OrderStatus.PLACED;
    Partner partner;

    Order(String id, Customer c, Restaurant r, int itemTotal, int deliveryFee) {
        this.id = id; this.customer = c; this.restaurant = r;
        this.itemTotal = itemTotal; this.deliveryFee = deliveryFee;
    }

    int total() { return itemTotal + deliveryFee; }
}

interface DeliveryFee {
    int fee(Restaurant r, Customer c);
}

class DistanceFee implements DeliveryFee {                      // Rs 20 + Rs 5 per km
    public int fee(Restaurant r, Customer c) {
        return 20 + 5 * r.location.distanceTo(c.location());
    }
}

interface AssignmentStrategy {
    Optional<Partner> choose(Order order, List<Partner> free);
}

// Closest to the restaurant, then by name
class NearestPartner implements AssignmentStrategy {
    public Optional<Partner> choose(Order order, List<Partner> free) {
        Location r = order.restaurant.location;
        return free.stream().min(Comparator.comparingInt((Partner p) -> p.location.distanceTo(r))
                .thenComparing(p -> p.name));
    }
}

interface OrderListener {
    void onStatusChange(Order order);
}

// Collects messages; the driver prints them
class CustomerNotifier implements OrderListener {
    private final List<String> outbox = new ArrayList<>();

    public void onStatusChange(Order o) {
        if (o.status == OrderStatus.ACCEPTED) return;           // nothing to tell the customer yet
        outbox.add("-> " + o.customer.name() + ": order " + o.id + " "
                + o.status.name().toLowerCase().replace('_', ' '));
    }

    List<String> drain() {
        List<String> out = new ArrayList<>(outbox);
        outbox.clear();
        return out;
    }
}

// The front door; one lock for dispatch
class FoodService {
    private final Map<String, Restaurant> restaurants = new HashMap<>();
    private final Map<String, Customer> customers = new HashMap<>();
    private final List<Partner> partners = new ArrayList<>();
    private final Map<String, Order> orders = new HashMap<>();
    // Accepted, no partner yet, oldest first
    private final Deque<Order> waiting = new ArrayDeque<>();
    private final DeliveryFee feeRule;
    private final AssignmentStrategy assignment;
    private final List<OrderListener> listeners;
    private int orderCount;

    FoodService(DeliveryFee feeRule, AssignmentStrategy assignment, List<OrderListener> listeners) {
        this.feeRule = feeRule; this.assignment = assignment; this.listeners = listeners;
    }

    synchronized void addRestaurant(String n, Location l) { restaurants.put(n, new Restaurant(n, l)); }
    synchronized void addItem(String r, String item, int price) { restaurant(r).menu.put(item, price); }
    synchronized void addCustomer(String n, Location l)    { customers.put(n, new Customer(n, l)); }
    synchronized void addPartner(String n, Location l)     { partners.add(new Partner(n, l)); }

    synchronized Order place(String customerName, String restaurantName,
                             Map<String, Integer> items) {
        Customer c = customers.get(customerName);
        if (c == null) throw new IllegalArgumentException("Unknown customer " + customerName);
        Restaurant r = restaurant(restaurantName);
        int itemTotal = 0;
        for (Map.Entry<String, Integer> e : items.entrySet()) {
            Integer price = r.menu.get(e.getKey());
            if (price == null) throw new IllegalArgumentException(r.name + " has no " + e.getKey());
            if (e.getValue() < 1) throw new IllegalArgumentException("Quantity must be at least 1");
            itemTotal += price * e.getValue();
        }
```

```
        Order o = new Order("O" + (++orderCount), c, r, itemTotal, feeRule.fee(r, c));
        orders.put(o.id, o);
        return o;
    }

    // Returns the partner, or empty if the order now waits for one.
    synchronized Optional<Partner> accept(String id) {
        Order o = move(id, OrderStatus.ACCEPTED, "accept");
        Optional<Partner> p = assign(o);
        if (p.isEmpty()) waiting.addLast(o);
        return p;
    }

    synchronized Order reject(String id) { return move(id, OrderStatus.REJECTED, "reject"); }

    synchronized Order pickUp(String id) {
        Order o = order(id);
        if (o.status == OrderStatus.ACCEPTED && o.partner == null)
            throw new IllegalStateException(id + " has no partner yet");
        return move(id, OrderStatus.PICKED_UP, "pick up");
    }

    // Returns any waiting order that got the freed partner.
    synchronized Optional<Order> deliver(String id) {
        Order o = move(id, OrderStatus.DELIVERED, "deliver");
        // The partner is now at the customer's door
        o.partner.location = o.customer.location();
        return release(o.partner);
    }

    synchronized Optional<Order> cancel(String id) {
        Order o = move(id, OrderStatus.CANCELLED, "cancel");
        waiting.remove(o);
        Partner p = o.partner;
        o.partner = null;
        return p == null ? Optional.empty() : release(p);
    }

    synchronized Order order(String id) {
        Order o = orders.get(id);
        if (o == null) throw new IllegalArgumentException("Unknown order " + id);
        return o;
    }

    private Order move(String id, OrderStatus next, String verb) {
        Order o = order(id);
        if (!o.status.canMoveTo(next))
            throw new IllegalStateException("Cannot " + verb + " " + id + ": it is " + o.status);
        o.status = next;
        listeners.forEach(l -> l.onStatusChange(o));
        return o;
    }

    // Choose and mark busy: one step, under the lock
    private Optional<Partner> assign(Order o) {
        List<Partner> free = partners.stream().filter(p -> p.current == null).toList();
        Optional<Partner> chosen = assignment.choose(o, free);
        chosen.ifPresent(p -> { p.current = o; o.partner = p; });
        return chosen;
    }

    // Free the partner, then serve the queue
    private Optional<Order> release(Partner p) {
        p.current = null;
        Order next = waiting.pollFirst();
        if (next == null) return Optional.empty();
        assign(next);
        return Optional.of(next);
    }

    private Restaurant restaurant(String name) {
        Restaurant r = restaurants.get(name);
        if (r == null) throw new IllegalArgumentException("Unknown restaurant " + name);
        return r;
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        CustomerNotifier notifier = new CustomerNotifier();
        FoodService service =
                new FoodService(new DistanceFee(), new NearestPartner(), List.of(notifier));
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "RESTAURANT" -> {
                        service.addRestaurant(p[1], loc(p[2], p[3]));
                        System.out.println("Restaurant " + p[1] + " at " + loc(p[2], p[3]));
                    }
                    case "ITEM" -> {
                        service.addItem(p[1], p[2], Integer.parseInt(p[3]));
                        System.out.println(p[1] + " menu: " + p[2] + " Rs " + p[3]);
                    }
                    case "CUSTOMER" -> {
                        service.addCustomer(p[1], loc(p[2], p[3]));
                        System.out.println("Customer " + p[1] + " at " + loc(p[2], p[3]));
                    }
```

```
                    case "PARTNER" -> {
                        service.addPartner(p[1], loc(p[2], p[3]));
                        System.out.println("Partner " + p[1] + " at " + loc(p[2], p[3]));
                    }
                    case "ORDER" -> {
                        Map<String, Integer> items = new LinkedHashMap<>();
                        for (int i = 3; i < p.length; i++) {
                            String[] kv = p[i].split(":");
                            if (kv.length != 2)
                                throw new IllegalArgumentException("Bad item " + p[i]);
                            items.put(kv[0], Integer.parseInt(kv[1]));
                        }
                        Order o = service.place(p[1], p[2], items);
                        System.out.println(o.id + " placed: " + o.customer.name() + " from "
                                + o.restaurant.name + ", items Rs " + o.itemTotal + " + delivery Rs "
                                + o.deliveryFee + " = Rs " + o.total());
                    }
                    case "ACCEPT" -> {
                        Optional<Partner> partner = service.accept(p[1]);
                        Order o = service.order(p[1]);
                        System.out.println(partner.map(x -> p[1] + " accepted, partner " + x.name
                                + " is "
                                + x.location.distanceTo(o.restaurant.location) + " km away")
                                .orElse(p[1] + " accepted, waiting for a partner"));
                    }
                    case "REJECT" -> {
                        Order o = service.reject(p[1]);
                        System.out.println(o.id + " rejected, refund Rs " + o.total());
                    }
                    case "PICKUP" -> {
                        Order o = service.pickUp(p[1]);
                        System.out.println(o.id + " picked up by " + o.partner.name);
                    }
                    case "DELIVER" -> {
                        Order o = service.order(p[1]);
                        Optional<Order> next = service.deliver(p[1]);
                        System.out.println(o.id + " delivered by " + o.partner.name);
                        next.ifPresent(Main::printAssigned);
                    }
                    case "CANCEL" -> {
                        Optional<Order> next = service.cancel(p[1]);
                        int refund = service.order(p[1]).total();
                        System.out.println(p[1] + " cancelled, refund Rs " + refund);
                        next.ifPresent(Main::printAssigned);
                    }
                    case "TRACK" -> {
                        Order o = service.order(p[1]);
                        String who = o.partner == null ? "" : ", partner " + o.partner.name;
                        System.out.println(o.id + ": " + o.status + who);
                    }
                    // N orders accepted at the same moment
                    case "RUSH" -> {
                        int n = Integer.parseInt(p[4]);
                        List<String> ids = new ArrayList<>();
                        for (int i = 0; i < n; i++)
                            ids.add(service.place(p[1], p[2], Map.of(p[3], 1)).id);
                        int got = rush(n, i -> service.accept(ids.get(i)).isPresent());
                        System.out.println("Rush: " + got + " got a partner, " + (n - got)
                                + " waiting");
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
            notifier.drain().forEach(System.out::println);
        }
    }

    static Location loc(String x, String y) {
        return new Location(Integer.parseInt(x), Integer.parseInt(y));
    }

    static void printAssigned(Order o) {
        System.out.println(o.id + " assigned to " + o.partner.name + ", "
                + o.partner.location.distanceTo(o.restaurant.location) + " km away");
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

**How to read it:** every status change goes through `move`, so the life cycle and the notifications can't be bypassed. `assign` and `release` are private and only called under the service's lock. After a delivery, the partner's location becomes the customer's, which is why Kiran is 7 km from Vaishali for O3.

## Part C — model answers

**1. Surge pricing.** A `SurgeFee` decorator around `DistanceFee` that doubles the fee when the waiting queue has more than 5 orders. It needs to see the queue length, so give it a small read-only view (a `Supplier<Integer>`), not the whole service. Order code doesn't change.

**2. Two orders per partner.** A partner gets a capacity and a list of orders instead of one. Assignment considers busy partners too: one already going to a nearby restaurant, with spare capacity and a route that doesn't add much distance. This is mostly a new `AssignmentStrategy` plus `Partner` changes; the order life cycle stays the same.

**3. A partner goes offline.** Partners send a heartbeat (every few seconds). A scheduled job marks a partner offline after, say, 2 minutes of silence. If they had an order that wasn't picked up, unassign it and put it back at the *front* of the waiting queue; if it was picked up, alert support, because the food is with them.

**4. Fifty cities, a million orders a day.** Split by city: dispatch in one city never needs another city's partners, so each city's dispatcher (and lock) is independent. Partner locations go into a store built for nearby-search (Redis geo commands, or a grid of cells) instead of scanning every partner. Orders are rows in a database, with status changes as conditional updates (`WHERE status = 'PLACED'`). Notifications go through a message queue so a slow SMS provider never slows ordering.

**5. "Use the State pattern."** "You'd be right if each status behaved differently; for the vending machine it did. Here every move does the same three things and only the allowed moves differ, so they fit in one table: `canMoveTo`. If statuses start to behave differently, say PICKED_UP starts live tracking and charges a cancellation fee, I'd move to State classes then."

**Common mistakes**

- Status changes scattered through the code instead of one method that checks the allowed moves.
- Choosing a free partner and marking them busy in two separately locked steps.
- Forgetting the waiting queue when a partner is freed by a cancellation.
- Leaving the partner's location at the start after a delivery.
- Sending notifications from inside order logic, or letting a failed notification undo a status change.

## Score yourself (1 point each)

| | | | |
|---|---|---|---|
| ☐ | An enum life cycle with allowed moves | ☐ | One method makes every status change |
| ☐ | Fee and assignment are strategies | ☐ | Notifications are listeners |
| ☐ | Nearest free partner, ties by name | ☐ | Choose and mark busy under one lock |
| ☐ | A freed partner takes the oldest waiting order | ☐ | Your output matches exactly, RUSH included |
| ☐ | Part A 2: you can say why not State | ☐ | Part C 4: you split by city |

# Back pages

## Warm-up answers

**W1 — Cache with TTL. Lazy** expiry checks the time when a key is read, so an expired value is never returned; **active** expiry sweeps regularly, so entries nobody reads don't fill memory. Real caches use both. An access-order `LinkedHashMap` gives the least recently used (its first entry); a `TreeSet` by expiry time (with a sequence number for ties) gives the soonest to expire. A test passes in a clock it can move. When full: an expired entry goes first, otherwise the least recently used.

**W2 — Meeting rooms.** [s1, e1) and [s2, e2) clash exactly when **s1 < e2 and s2 < e1**. `lowerEntry(end)` finds the last meeting starting before your end; because a room's meetings never overlap, it's the only one that can reach into your slot. The room's own lock covers "check, then add". "Any room" tries rooms in order, each try already safe, moving on when one is busy.

| # | W3 answer | Why |
|---|---|---|
| 1 | **Strategy** | scoring rules that differ per exam, with more coming. |
| 2 | **State** | several operations, each depending on the attempt's status. |
| 3 | **Adapter** | a vendor's SDK we can't change, behind our own interface. |
| 4 | **Observer** | reactions to one event, with more to come. |
| 5 | **Chain of Responsibility** | up to X this one decides; otherwise pass it on. |
| 6 | **Facade** | several steps in order, behind one call, used from three places. |
| 7 | **Composite** | a whole and its parts answer the same question. |
| 8 | **Nothing**: a scheduled job | "hasn't paid in 3 days" isn't an event. |
| 9 | **Flyweight** | the same unchanging data shared by thousands of objects. |
| 10 | **Proxy** | the same interface, with a cache and an access check in front. |
| 11 | **Nothing**: a record | two fields that never change. |
| 12 | **Builder** | required parts plus many optional ones, never changing afterwards. |

Both designs right and 10 or more of the 12: on track. If W1 or W2 was shaky, re-do that problem's Part A from PDF 4 before Problem 2 here.

## Mistake log

One row per mistake, in your own words. Before starting PDF 6, read this page and the earlier logs once.

| Date | Problem | What I did | What's right | Rule to remember |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

## Re-do schedule

| Problem | Attempted (time taken) | Day 3: Part A from memory (15 min) | Day 10: Part A + the core class | Day 30: the whole problem, timed | Score /10 |
|---|---|---|---|---|---|
| Logging Framework | | | | | |
| Hotel Booking | | | | | |
| Task Scheduler | | | | | |
| Food Delivery | | | | | |

The core class to rebuild on day 10: `LogManager` with the level walk · `Hotel.book` with the per-room lock · `Scheduler.runUntil` · `FoodService`'s `move`, `assign` and `release`. PDF 4's day-30 re-dos are probably due around now too.

**Next: PDF 6, the last one.** Traffic Signal / Amazon Locker, a Notification System, Inventory Management and Ride-sharing. Its warm-up will ask you to re-design the Task Scheduler and Food Delivery from memory, so the day-10 re-dos above are worth doing before you open it.